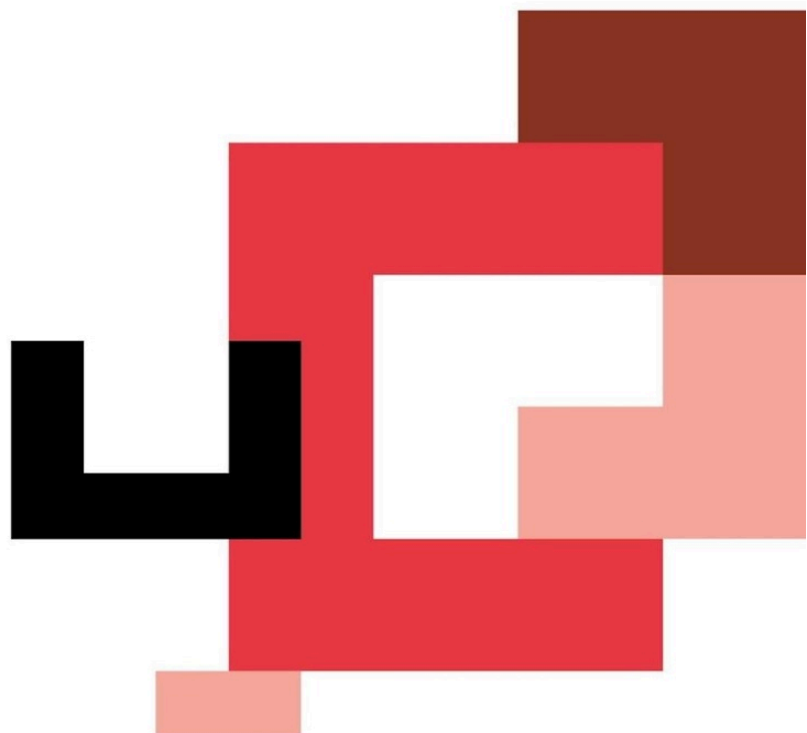




FULL STACK DEVELOPER + IA

Módulo 3 | Feature 1

Fundamentos React + UI base
del e-commerce



Objetivo de la feature

Construir la base del frontend con React, entendiendo:

- Qué es React y cómo se basa en componentes
- Cómo estructurar una aplicación real (pages + components)
- Cómo pasar datos con **props**
- Cómo gestionar estado básico con **useState**
- Cómo navegar entre páginas con **react-router-dom**
- Cómo construir una **UI completa de catálogo**

Importante

Esta feature (la tienda) se construye en **Sprint 1 + Sprint 2**.

En este sprint nos centramos en la **UI y estructura**.

Si no llegas a todo, no pasa nada.

En el Sprint 2 continuaremos construyendo la UI mientras conectamos la API.

Aquí tienes el ejemplo que puedes tomar como guía. Navega por la página:

<https://shop-example-react.netlify.app/sprint-1/>

Requisitos

Tech / configuración

- Node.js 18+
- Proyecto con Vite
- React
- react-router-dom

```
Shell
npm create vite@latest

npm install

npm install react-router-dom

npm run dev
```

Estructura sugerida

Shell

src/

styles/

variables.css

index.css

data/

mockProducts.js

components/

Layout/

Header/

Footer/

ProductCard/

ProductGrid/

pages/

HomePage/

ProductsPage/

ProductDetailPage/

NotFoundPage/

router/

index.jsx

App.jsx

main.jsx

Rutas

Método	Ruta	Descripción
GET	/	Home
GET	/products	Catálogo
GET	/products/:id	Detalle
GET	*	404

Qué hay que hacer (paso a paso)

1) Crear el proyecto

- Crear proyecto con Vite
- Limpiar archivos innecesarios
- Crear estructura de carpetas

Objetivo: base limpia y escalable.

2) Configurar estilos globales

- Crear `variables.css`
- Crear `index.css`
- Importar en `main.jsx`

Objetivo: consistencia visual desde el inicio.

3) Crear datos mock

Crear `mockProducts.js` con:

```
JavaScript
{
  id,
  title,
  price,
  image,
  description
}
```

Objetivo: simular backend real (mismo shape que API futura).

Aquí tienes un ejemplo de cómo podría quedar:

```
JavaScript
export const MOCK_PRODUCTS = [
  {
    id: '1',
    name: 'Camiseta Básica',
    description: 'Camiseta de algodón 100%, cómoda y duradera. Disponible en varios colores.',
    price: 19.99,
    stock: 50,
    imageUrl: 'https://placeholder.co/400x400?text=Camiseta',
    createdAt: '2024-01-01T00:00:00.000Z',
  }, ...
]
```

Puedes usar la IA para que te cree por lo menos 10 productos.

Objetivo: simular backend real (mismo shape que API futura).

4) Crear Layout y estructura base

Layout

- Header
- Footer
- `<Outlet />`

Header

- Navegación
- Menú móvil (`useState`)

Footer

- Información básica

Objetivo: estructura global de la app.

5) Configurar routing

En `router/index.jsx`:

- `createBrowserRouter`
- Layout como wrapper
- Definir rutas

Uso de:

- `Link`
- `NavLink`
- `useParams`
- `Outlet`

Objetivo: navegación completa.

6) Crear páginas

HomePage

- Mostrar productos destacados

ProductsPage

- Mostrar todos los productos
- Buscador con `useState`

ProductDetailPage

- Obtener id con `useParams`
- Mostrar detalle
- Contador de cantidad

NotFoundPage

- Página 404

Objetivo: primeras vistas reales del e-commerce.

7) Crear componentes de producto

ProductCard

- Recibe `product` por props

ProductGrid

- Recibe `products`
- Render con `.map()`
- key obligatoria

Objetivo: reutilización + render dinámico.

Uso de la IA en este sprint

Para guiarte

- Explicación de React (componentes, props, estado)
 - Revisión de estructura
 - Debug de errores
-

Prompts útiles

Componentes

None

Estoy organizando una app en React con Layout, Header, Footer y ProductCard.

¿Está bien separada la responsabilidad? ¿Qué mejoraría?

Routing

None

Explicame cómo funciona createBrowserRouter y Outlet en React Router.

Debug

None

Tengo este error al renderizar una lista en React.

Te paso el código. Dime las causas más probables.

Pistas

Props

Los datos bajan de padre a hijo.

useState

Se usa para:

- Menú móvil
- Buscador
- Contador

useParams

JavaScript

```
const { id } = useParams()
```


Listas

JavaScript

```
products.map(product => ...)
```

Siempre con:

JavaScript

```
key={product.id}
```

Importante que tienen que tener en cuenta

Este sprint es visual

No hay backend real todavía.

No sobrecomplicar

No hace falta lógica avanzada.

Separación clara

- components → reutilizables
- pages → vistas

La UI no tiene que estar perfecta

Lo importante es la **estructura**, no el pixel perfect.
En el Sprint 2 se seguirá trabajando esta UI.

Routing correcto

Si el routing falla → la app no funciona.

Checks rápidos (autoevaluación)

- npm run dev funciona
 - Navegación entre páginas funciona
 - /products muestra productos
 - /products/:id muestra detalle
 - Buscador funciona
 - 404 funciona
-

Qué pasará en el Sprint 2

- Eliminamos mockProducts
 - Conectamos con API real (<http://localhost:3000>)
 - Introducimos **Axios**
 - Introducimos **useEffect**
 - Creamos **custom hooks**
 - Añadimos **login y registro**
 - Seguimos construyendo UI
-

Enfoque del sprint

- Construir base sólida
- Entender React
- No bloquearse en detalles

Avanza lo máximo posible, pero sin frustrarte si no llegas a todo.
Este sprint se completa entre Sprint 1 y Sprint 2.